
NOTE

A simple programming trick for determining whether an integer is odd or even uses the modulus operator (%). Any even integer mod 2 results in 0, and any odd integer mod 2 results in 1.

Those are the recursive cases, but what are the base cases? Mathematically speaking, any number to the zeroth power is defined as 1, while any number to the first power is the number itself. So for any function call `exponentByRecursion(a, n)`, if `n` is 0 or 1, we can simply return 1 or `a`, respectively, because a_0 is always 1 and a_1 is always `a`.

Using all this information, we can write code for the `exponentByRecursion()` function. Here is an *exponentByRecursion.py* file with the Python code:

Python

```
def exponentByRecursion(a, n):
    if n == 1:
        # BASE CASE
        return a
    elif n % 2 == 0:
        # RECURSIVE CASE (When n is even.)
        result = exponentByRecursion(a, n // 2)
        return result * result
    elif n % 2 == 1:
        # RECURSIVE CASE (When n is odd.)
        result = exponentByRecursion(a, n // 2)
        return result * result * a

print(exponentByRecursion(3, 6))
print(exponentByRecursion(10, 3))
print(exponentByRecursion(17, 10))
```

And here is the equivalent JavaScript code in *exponentByRecursion.html*:

JavaScript

```
<script type="text/javascript">
function exponentByRecursion(a, n) {
    if (n === 1) {
        // BASE CASE
        return a;
    }
}
```
